

Практическая работа №6: Корутины в Kotlin

Тема: Основы асинхронного программирования с использованием корутин Kotlin.

Цель:

- Научиться создавать и управлять корутинами.
 - Понять разницу между синхронным и асинхронным выполнением кода.
 - Освоить использование функций `launch`, `async`, `withContext` и обработку исключений в корутинах.
 - Получить навыки работы с диспетчерами (`Dispatchers`).
-

Теоретическая часть (краткое изложение):

1. **Корутины** — это легковесные потоки, которые позволяют писать асинхронный код в последовательном стиле.
 2. **Структурированный параллелизм** обеспечивает отмену и распространение исключений через иерархию корутин.
 3. **Функции-строители корутин:**
 - `launch` — запускает корутину без возврата результата.
 - `async` — возвращает `Deferred<T>`, который можно ожидать с помощью `await()`.
 4. **Диспетчеры** определяют поток выполнения:
 - `Dispatchers.Main` — для работы с UI.
 - `Dispatchers.IO` — для сетевых и файловых операций.
 - `Dispatchers.Default` — для CPU-интенсивных задач.
 5. **Приостанавливаемые функции** (`suspend`) могут быть остановлены и возобновлены без блокировки потока.
-

Практический пример:

```
import kotlinx.coroutines.*

fun main() = runBlocking {
    println("Start: ${Thread.currentThread().name}")

    val job = launch {
        withContext(Dispatchers.IO) {
            delay(1000)
            println("IO Work: ${Thread.currentThread().name}")
        }
        println("After IO: ${Thread.currentThread().name}")
    }

    job.join()
}
```

```
println("End: ${Thread.currentThread().name}")  
}
```

Вывод:

```
Start: main @coroutine#1  
IO Work: DefaultDispatcher-worker-1 @coroutine#1  
After IO: main @coroutine#1  
End: main @coroutine#1
```

Ход работы:

1. Создайте новый проект в IntelliJ IDEA с поддержкой Kotlin.
2. Добавьте зависимость в `build.gradle.kts`:

```
implementation("org.jetbrains.kotlinx:kotlinx-coroutines-core:1.7.3")
```

3. Выполните задание согласно вашему варианту (см. ниже).
4. Сделайте скриншоты работы программы и сохраните в папку `images`.
5. Оформите `README.md` по шаблону.
6. Загрузите проект в репозиторий Gogs.

Варианты заданий (выбор по номеру в журнале):

1. Написать программу, которая запускает две корутины с задержкой 1 сек и выводит их результаты последовательно.
2. Использовать `async` и `await` для параллельного выполнения двух сетевых запросов (заглушек).
3. Написать код, где одна корутина отменяется через 500 мс, а другая завершается успешно.
4. Реализовать обработку исключения в корутине с помощью `try-catch`.
5. Использовать `withContext` для переключения между `Dispatchers.Main` и `Dispatchers.IO`.
6. Создать цепочку корутин, где ошибка в дочерней корутине отменяет родительскую.
7. Написать программу, имитирующую загрузку данных с прогресс-баром в консоли.
8. Реализовать параллельную обработку списка чисел с использованием `async`.
9. Использовать `coroutineScope` для организации структурированного параллелизма.
10. Написать код, где корутина ожидает завершения нескольких других корутин с помощью `joinAll`.
11. Реализовать повторный запуск корутины при возникновении ошибки.
12. Использовать `SupervisorJob` для независимого выполнения корутин.
13. Написать программу, которая использует `Channel` для обмена данными между корутинами.
14. Реализовать таймаут для корутины с помощью `withTimeout`.
15. Создать аналог «погодного приложения» с параллельным получением прогноза и температуры.

Пример задания (вариант 1):

Задание:

Написать программу, которая выводит "Start", затем запускает корутину с задержкой 2 сек, выводит "Coroutine finished", и затем "End".

Код:

```
import kotlinx.coroutines.*

fun main() = runBlocking {
    println("Start")
    launch {
        delay(2000)
        println("Coroutine finished")
    }.join()
    println("End")
}
```

Вывод:

```
Start
Coroutine finished
End
```

Критерии оценки:

- **5 (отлично):** Программа работает корректно, код хорошо структурирован, есть обработка ошибок, README оформлен полностью.
- **4 (хорошо):** Программа работает, но есть незначительные недочеты в коде или оформлении.
- **3 (удовлетворительно):** Программа работает частично, есть ошибки в логике или оформлении.
- **2 (неудовлетворительно):** Программа не работает, код не соответствует заданию.

Контрольные вопросы:

1. Что такое корутина и чем она отличается от потока?
2. Какие диспетчеры вы знаете и для чего они используются?
3. Что такое `Deferred` и как получить его результат?
4. Как обрабатываются исключения в корутинах?
5. Что такое «структурированный параллелизм»?

Файловая структура репозитория:

```
praktika10-coroutines/
├─ src/
```

```
├── main/
│   └── kotlin/
│       └── Main.kt
├── images/
│   ├── screenshot1.png
│   └── screenshot2.png
├── .gitignore
└── README.md
```

Содержание README.md:

Практическая работа №10: Корутины в Kotlin

Вариант X: [Название задания]

Задание:

[Текст задания]

Код:

Скриншоты работы программы:

![[Screenshot]](images/screenshot1.png)